"Alumni Connect: A Student–Alumni Networking Platform Using Data Structures and Algorithms"

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

M.V.L SAMHITA	**2520030509**
P. MANASVI	**2520030072**
B. VIDA SWETHA	**2520030100**

Team Number: 19
Section: 8

Under the Guidance of

Dr. V. Sireesha
Professor
Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"Alumni Connect: A Student–Alumni Networking Platform Using Data Structures and Algorithms"** is a record of Bonafide work of **M.V.L Samhita – 2520030509, P. Manasvi – 2520030072, B. Vida Swetha – 2520030100** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

M.V.L Samhita – 2520030509
P. Manasvi – 2520030072
B. Vida Swetha – 2520030100

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled **"Alumni Connect: A Student–Alumni Networking Platform Using Data Structures and Algorithms"** is a bonafide work done and submitted by **M.V.L SAMHITA – 2520030509, P. MANASVI – 2520030072, B. VIDA SWETHA – 2520030100** in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27.**

Signature of the Guide
(Professor V. Sireesha)

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our guide, Dr. V. Sireesha, Professor, Department of Computer Science and Engineering, for her valuable guidance, constant encouragement and support throughout the development of this project. Her suggestions helped us understand how data structures and algorithms can be applied to solve a real problem.

We are thankful to Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the facilities and a supportive environment to carry out this work.

We also thank the faculty members of the department, our friends and our families for their help and encouragement during the course of this project.

M.V.L Samhita – 2520030509
P. Manasvi – 2520030072
B. Vida Swetha – 2520030100

Abstract

Students often find it hard to reach alumni who can guide them about careers, companies and skills, because alumni details are scattered across social media, emails and personal contacts. This project, Alumni Connect, is a web-based student–alumni networking platform that brings both groups together in one place. Its main aim is to let a student search for alumni quickly and send a connection request, and to let an alumnus manage those requests easily.

The platform has two roles, Student and Alumni, each with its own dashboard, profile and connection pages. Students can search the alumni directory by name, organization, skills, interests or bio, view alumni profiles, send connection requests and see their recent searches. Alumni can update their profiles and accept or reject incoming requests. Registration, login with a captcha and a security code, and role-based page protection keep each role limited to its own pages.

Alumni Connect primarily demonstrates Java programming and Data Structures and Algorithms. Its Java 17+ backend implements Trie-based prefix suggestions, KMP and Aho-Corasick keyword searching, Edit Distance for typo suggestions, Longest Common Subsequence (LCS) for profile similarity, and Insertion Sort for ranking. It also demonstrates graph algorithms: Dijkstra for shortest connection paths, Floyd–Warshall for network analysis, and bipartite matching for mentor allocation. The Java server processes an in-memory snapshot of the sample data; persistent database storage is not included.

Table of Contents

Ch. No.	Title	Contents	Page No.
	Acknowledgement		
	Abstract		
	Table of Contents		
	List of Figures		
1	Introduction	1.1 Overview 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives	1
2	System Requirements	2.1 Functional Requirements 2.2 Non-Functional Requirements 2.3 Hardware and Software Requirements	2
3	System Design	3.1 High-level Architecture Diagram 3.2 Description of Each Layer 3.3 Connection State Machine	3
4	Implementation	4.1 Module-wise Implementation 4.2 Front-end Logic 4.3 Data Storage 4.4 DSA Implementation	5
5	Results	5.1 Algorithm Test Results 5.2 Feature Testing	7
6	Challenges & Limitations	6.1 Issues Faced During Development 6.2 Solutions Applied 6.3 Limitations of the System	8
7	Conclusion & Future Work	7.1 Conclusion 7.2 Future Work 7.3 Skills Learned During Development	9
	References		10

List of Figures

3.1 Layered Architecture of Alumni Connect 3
3.2 Connection Request State Machine 4
3.3 Search Flow in the Alumni Directory 5

1. INTRODUCTION

1.1 Overview

Alumni Connect is a web application that connects current students of a college with its graduated alumni. A student can search the alumni directory, read an alumnus profile and send a connection request. The alumnus can then accept or reject the request. Once accepted, both sides can see the connection in their portals.

The main technical focus of Alumni Connect is its Java 17+ backend, which uses the JDK built-in HTTP server to expose API endpoints for the DSA features. The DSA and graph algorithms are implemented in Java under backend/src/alumniconnect/. Sample user, profile and connection data is supplied to the backend as an in-memory snapshot for algorithm calls. The search flow is illustrated in Fig. 3.3.

1.2 Problem Statement

Students who want career advice have no single place to find alumni. Alumni information is spread across different platforms, and a simple search often fails when the student makes a spelling mistake, uses more than one keyword or does not know the exact name. There is also no easy way to manage requests between students and alumni. The problem is to design a simple platform where a student can find the right alumnus quickly using efficient search algorithms, and where connections can be requested and managed in an organised way.

1.3 Scope of the Project

- Two user roles only: Student and Alumni.
- Registration, login with captcha and security code, and role-based access to pages.
- Alumni directory with search, autocomplete, typo suggestions, filters and sorting.
- Connection requests with Pending, Connected and Rejected states.
- Student and alumni profiles, and a recent searches page for students.
- A "How It Works" page that demonstrates every algorithm interactively.

The current supplied version includes a Java backend for algorithm and graph API calls, but does not use a persistent database. Real email delivery and chat/messaging between users are not included.

1.4 Objectives

- To build a single platform where students and alumni can connect.
- To implement the core data structures and algorithms of the course from scratch, without using built-in sort or search functions.
- To use these algorithms in real features such as autocomplete, keyword search, typo correction, similar alumni and ranking.
- To make the application simple, responsive and easy to use on all devices.

2. SYSTEM REQUIREMENTS

2.1 Functional Requirements

The functions provided by the system for each role are listed in Table 2.1.

Table 2.1 Functional requirements

Role	Functional Requirements
Guest	View the home page and DSA demonstration page; register as a Student or Alumni; log in.
Student	View dashboard; search the alumni directory; use autocomplete and typo suggestions; filter by organization and graduation year; sort results; view alumni profile and similar alumni; send connection requests; view connections; view and clear recent searches; update own profile.
Alumni	View dashboard; update profile (company, skills, interests, bio); view incoming requests; accept or reject requests; view connected students.

2.2 Non-Functional Requirements

- **Performance: The Java algorithms operate on small in-memory data sets, making searches and demonstrations responsive.**
- **Security:** Pages are protected by role checks, and login needs a captcha and a security code.
- **Responsiveness:** The layout adapts to mobile, tablet and desktop screens.
- **Usability:** The interface is simple, with clear navigation and short toast messages for feedback.
- **Maintainability:** The code is divided into separate files for authentication, storage, data, navigation and algorithms.

2.3 Hardware and Software Requirements

Table 2.2 Hardware and software requirements

Item	Requirement
Hardware	Computer with JDK 17 or newer for running the Java backend
Software	Google Chrome, Microsoft Edge, Firefox or Safari
Languages	Java 17 or newer
Data storage	In-memory data structures and sample-data storage
Editor	Visual Studio Code

3. SYSTEM DESIGN

3.1 High-level Architecture Diagram

The application is organised in layers, as shown in Fig. 3.1. Each layer talks only to the layer next to it, which keeps the code easy to understand.

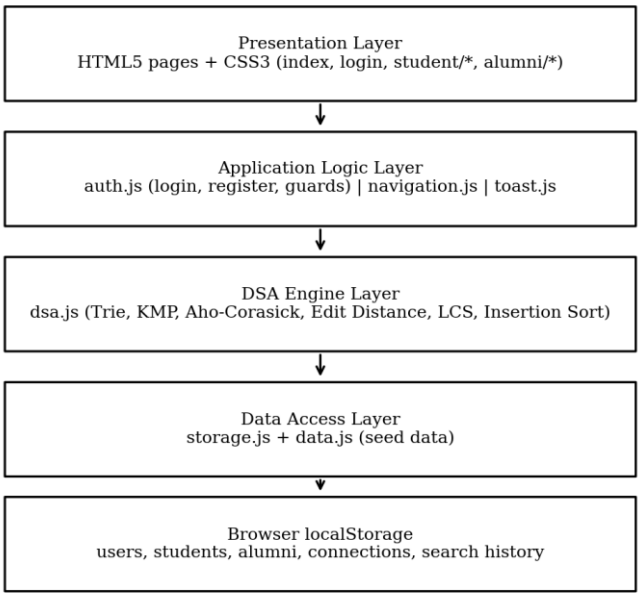


Fig. 3.1 Layered Architecture of Alumni Connect

3.2 Description of Each Layer

Table 3.1 Layers of the system

Layer	Description
Presentation	Provides the Student and Alumni views through which users access the Java-based features.
Application Logic	auth.js handles registration, login, captcha, security code and page guards. navigation.js handles the sidebar and mobile menu. toast.js shows messages.
DSA Engine	dsa.js contains DynamicArray, Queue, Naive search, KMP, Trie, Aho-Corasick, Edit Distance, LCS, Insertion Sort and Linear Search, plus a search facade used by the directory.
Data Access	storage.js reads and writes users, profiles, connections and search history. data.js loads sample alumni and student records on first run.
Storage	The Java backend processes an in-memory snapshot; persistent database storage is not included.

3.3 Connection State Machine

A connection between a student and an alumnus moves through the states shown in Fig. 3.2. The state is stored in one place, so both the student and the alumni portals always show the same status.

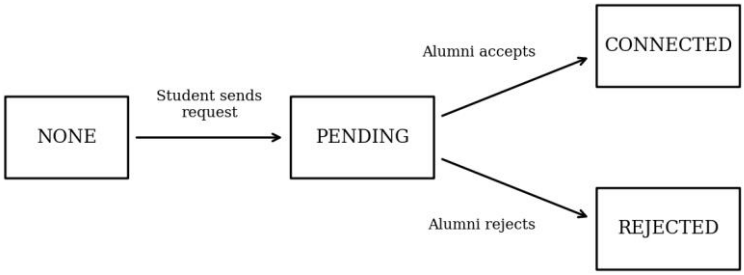


Fig. 3.2 Connection Request State Machine

4. IMPLEMENTATION

4.1 Module-wise Implementation

Table 4.1 Modules of the project

Module	Files	Work done
Authentication	Java authentication/API support and role-based workflows	Registers users, checks email, password and role, shows a captcha, asks for a security code and starts a session.
Student Portal	Student features: directory search, profiles, connections and recent searches	Dashboard, alumni directory with search, alumni profile view, connections, recent searches and profile.
Alumni Portal	Alumni features: profile updates, requests and connected students	Dashboard, profile editing, connection requests and connected students.
Storage and Data	storage.js, data.js	Saves and loads records; sample data of 10 alumni and 7 students is loaded on first run.
DSA Engine	Java DSA and graph algorithm classes	All algorithms and an interactive page to try them.
Navigation	navigation.js, toast.js	Sidebar, mobile menu, scroll effects and notification messages.

4.2 Front-end Logic

The application provides separate Student and Alumni views. The main technical processing is performed by Java algorithms, which search and rank alumni records and support connection-network analysis.

On the directory page, the search flow in Fig. 3.3 runs while the student types. The Trie gives suggestions, the edit distance function checks for a possible typo, and the search engine runs KMP for one word or Aho-Corasick for several words. Finally the results are sorted with Insertion Sort and shown as cards. Every search is saved in the student's search history.

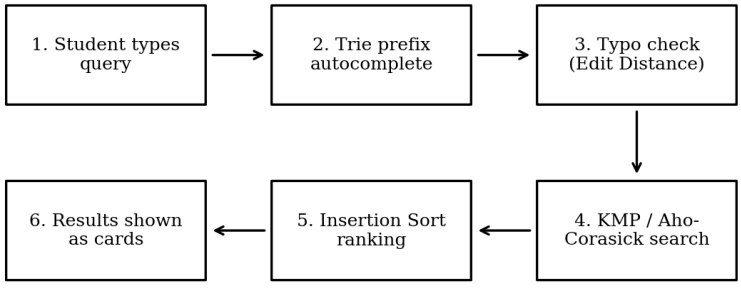


Fig. 3.3 Search Flow in the Alumni Directory

4.3 Data Storage

For this demonstration, sample user, profile, connection and search-history records are held in browser storage and supplied to the Java backend as needed. The backend keeps the received snapshot in memory only; the project does not include persistent database storage.

Table 4.2 Demonstration data storage keys

Storage key	Data stored
alumniConnect_users	Name, email, password and role of every user
alumniConnect_students	Student profile details
alumniConnect_alumni	Alumni profile: organization, graduation year, skills, interests, bio
alumniConnect_connections	Connection requests and their status
alumniConnect_searchHistory	Query, number of matches and algorithm used for each student search
alumniConnect_currentUser	Currently logged-in user

4.4 DSA Implementation

The core DSA and graph algorithms are implemented in Java under backend/src/alumniconnect/dsa/ and backend/src/alumniconnect/graph/. The required search and sorting algorithms are implemented directly rather than replaced with built-in sort/search methods. Table 4.3 summarises their use in Alumni Connect.

Table 4.3 DSA concepts and their use

Concept	Function / Class	Use in Alumni Connect
Dynamic Array	DynamicArray	Stores search results; capacity doubles when full
Queue	Queue	FIFO queue used while building Aho-Corasick links
Naive Search	naiveSearch()	Baseline character-by-character search for comparison
KMP Search	KMP.search(), computeLPSArray()	Single-keyword search over name, company, skills and bio
Trie and DFS	Trie, getSuggestions()	Autocomplete suggestions in the search bar
Aho-Corasick and BFS	AhoCorasick	Multi-keyword search; failure links built level by level using the Queue
Edit Distance (DP)	computeEditDistance()	"Did you mean" suggestion for typing mistakes
LCS (DP)	computeLCSLength()	Similar alumni score: 50% skills, 30% interests, 20% bio
Insertion Sort	insertionSort()	Sorting search results and similar alumni by score
Linear Search	linearSearch()	Looking up a record by ID
Dijkstra	Dijkstra.java, MinHeap.java, SocialGraph.java	Shortest path through accepted connections; path cost is number of hops.

Concept	Function / Class	Use in Alumni Connect
Floyd–Warshall	FloydWarshall.java	Overall network analysis: shortest distances, average distance and diameter.
Bipartite Matching	BipartiteMatching.java, MentorScore.java	Matches students with suitable alumni mentors using augmenting paths.

Time complexity of the main algorithms: KMP runs in O(n + m), the Trie prefix lookup takes O(L) for a word of length L, Edit Distance and LCS take O(m × n) with a 2D table, and Insertion Sort takes $O(n^2)$ in the worst case, which is acceptable for the small result lists used here.

5. RESULTS

5.1 Algorithm Test Results

Each algorithm in dsa.js was run with sample inputs and gave the expected output, as shown in Table 5.1.

Table 5.1 Algorithm test results

Algorithm	Input	Output
Naive Search	Text "Java Developer", pattern "Java"	Match at index 0 (14 comparisons)
KMP	Pattern "abab"	LPS array [0, 0, 1, 2]
KMP	Text "Java Developer at Google, Java", pattern "Java"	Matches at indices 0 and 26
Trie + DFS	Words Java, Java DSA, Java Algorithms; prefix "Ja"	Java, Java DSA
Aho-Corasick	Keywords java, react, aws on "Java and React developer with AWS skills"	All three keywords found at indices 0, 9 and 30
Edit Distance	"algoritm" and "algorithm"	Distance = 1
Typo Suggestion	"pythn" against [python, java]	Suggests "python"
LCS	"react node" and "node react"	LCS length = 5
Insertion Sort	[90, 60, 80, 70], descending	[90, 80, 70, 60]
Linear Search	ID "A003" in [A001, A002, A003]	Found at index 2 (3 comparisons)
Dynamic Array	Initial capacity 2, three items added	Size 3, capacity doubled to 4

5.2 Feature Testing

Table 5.2 Feature testing summary

Feature	Result
Registration and login	Users can register as Student or Alumni and log in with captcha and security code
Role-based access	A student cannot open alumni pages and an alumnus cannot open student pages; guests are sent to login
Directory search	Autocomplete, KMP and Aho-Corasick search, filters and sorting work together
Connection requests	A request sent by a student appears on the alumni requests page; accepting it updates both portals
Search history	Student searches are saved and can be repeated or cleared
Data persistence	Profile changes and connections remain after the page is reloaded
Responsiveness	Layout checked on mobile, tablet and desktop screen sizes

6. CHALLENGES & LIMITATIONS

6.1 Issues Faced During Development

- Writing the algorithms without built-in functions such as sort() and includes().
- Building the failure links of Aho-Corasick correctly using a queue.
- Keeping the student and alumni portals consistent when a connection status changes.
- Protecting pages without a server.

6.2 Solutions Applied

- Wrote our own Insertion Sort and Linear Search and tested them with small examples first.
- Drew the Trie and the failure links on paper, and then coded the BFS step by step.
- Kept all connection requests in one list with a status field, which both portals read.
- Added a guard function that checks the stored session and role on every protected page.

6.3 Limitations of the System

- The demonstration uses sample data and in-memory backend state, so it does not provide shared persistent data across devices or users.
- Passwords and the security code are handled on the client side, so this is a demonstration and not a production-level security system.
- Alumni records are sample data and are not verified.
- There is no chat or email notification between students and alumni.

7. CONCLUSION & FUTURE WORK

7.1 Conclusion

Alumni Connect demonstrates how Java and Data Structures and Algorithms can solve practical search and networking problems. Trie supports prefix suggestions; KMP and Aho-Corasick support keyword search; Edit Distance suggests corrections for typing mistakes; LCS helps compare profile similarity; and Insertion Sort ranks results. Graph algorithms—Dijkstra, Floyd–Warshall and bipartite matching—support connection-path analysis and mentor allocation. A lightweight Java 17+ HTTP server exposes the algorithm APIs. The demonstration uses sample data and in-memory backend state rather than a persistent database.

7.2 Future Work

- Add a backend server and a database so that data is shared between users and devices.
- Add proper password hashing and email-based verification.
- Add chat between connected students and alumni.
- Use faster sorting algorithms such as Merge Sort or Quick Sort for large data.
- Add alumni verification by the college and event announcements.

7.3 Skills Learned During Development

- Implementing Trie, KMP, Aho-Corasick, Edit Distance, LCS and sorting from scratch.
- Implementing Java-based data structures, searching, sorting, dynamic programming and graph algorithms.
- Understanding in-memory data handling, algorithm integration and role-based application workflows.
- Working as a team, dividing modules and testing each other's work.

REFERENCES

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009.
2. D. E. Knuth, J. H. Morris and V. R. Pratt, "Fast pattern matching in strings," SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977.
3. A. V. Aho and M. J. Corasick, "Efficient string matching: An aid to bibliographic search," Communications of the ACM, vol. 18, no. 6, pp. 333–340, 1975.
4. V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions and reversals," Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.
5. Oracle Java Documentation, Java Platform, Standard Edition (Java SE) documentation.
6. Oracle Java Documentation, Java Language and API documentation.
7. Project source code: Alumni Connect (Java DSA and graph algorithm classes under backend/src/alumniconnect/).

ALUMNICONNECT APPLICATION SCREENSHOTS

The screenshots below show the AlumniConnect demonstration, including the landing page, feature overview, authentication, student dashboard, directory search, typo suggestions and mentor matching. The Java backend and graph algorithms can be run from the backend folder using run.bat on Windows with JDK 17 or newer.

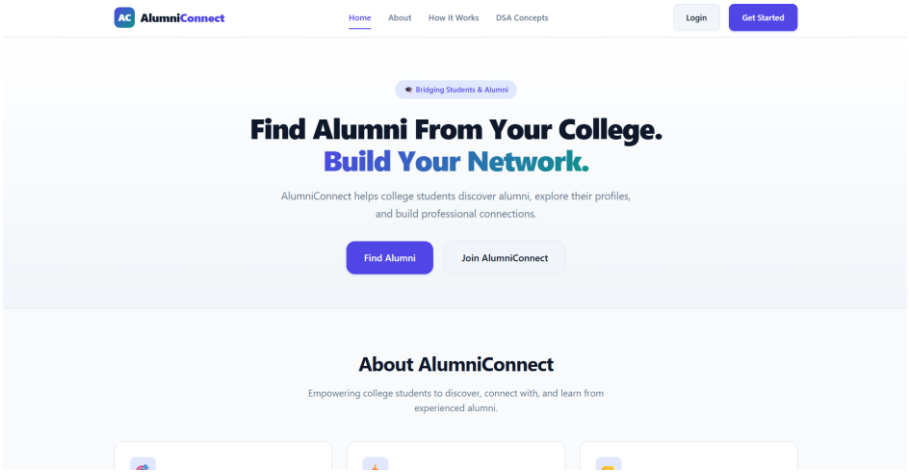


Figure A.1: AlumniConnect home page and project introduction

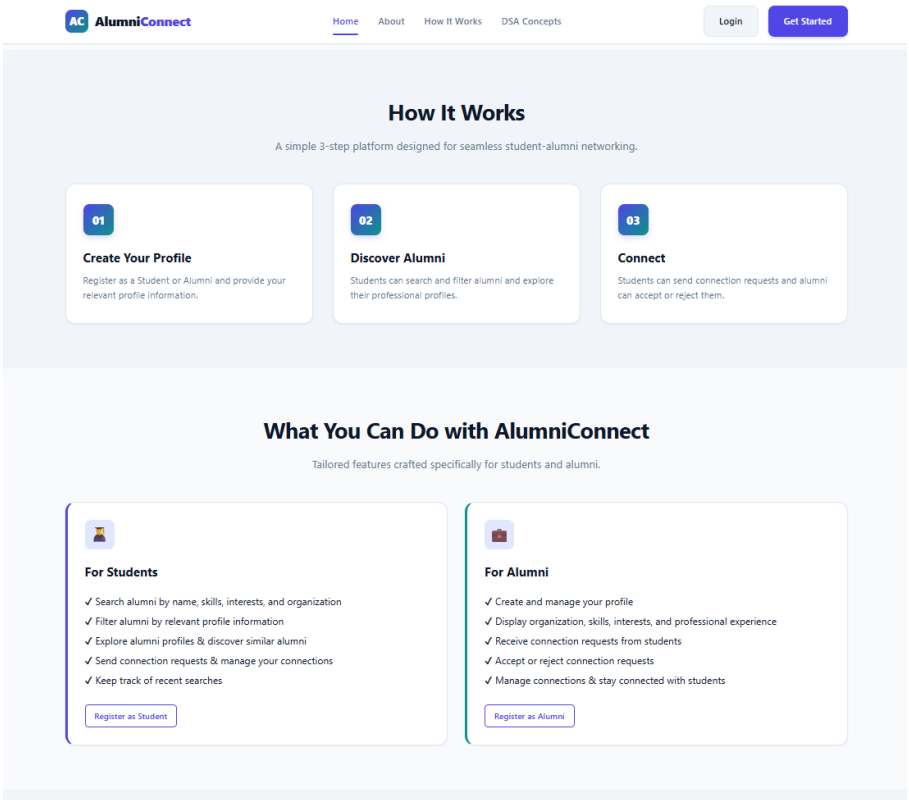


Figure A.2: How It Works and student/alumni feature overview

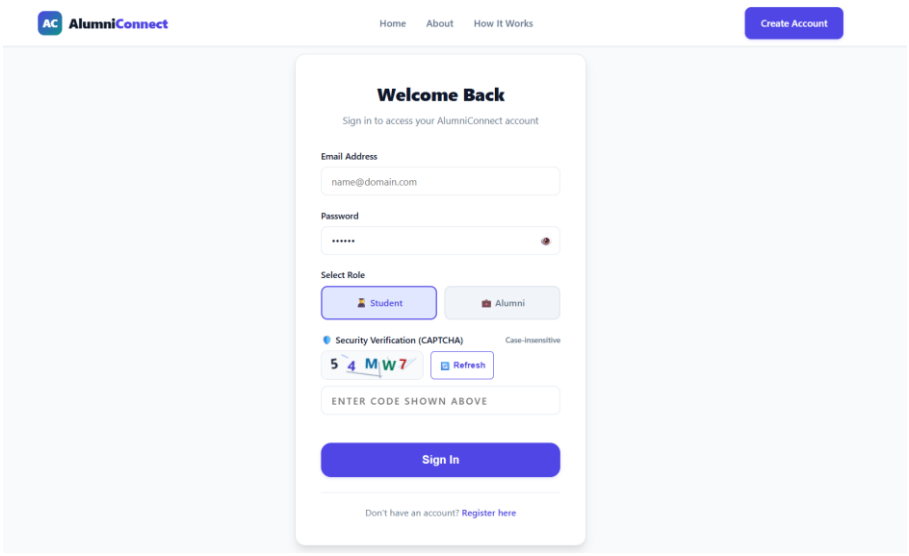


Figure A.3: Login page with role selection and CAPTCHA

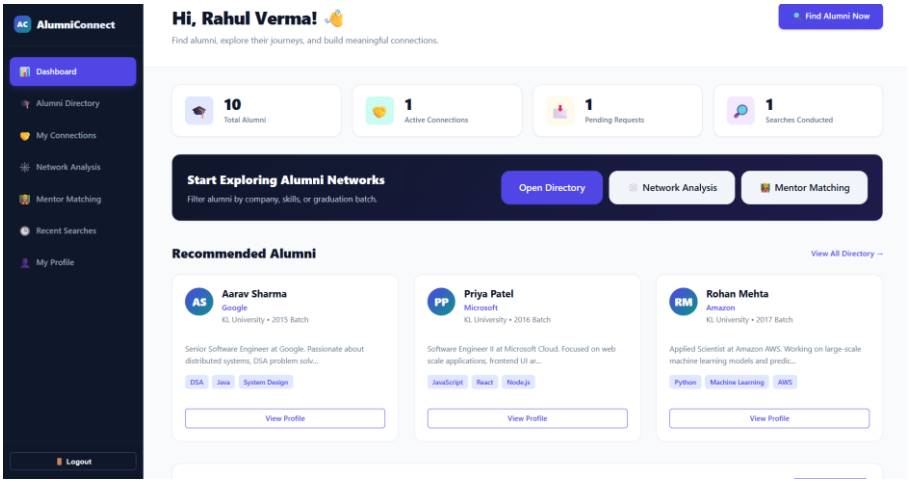


Figure A.4: Student dashboard and recommended alumni

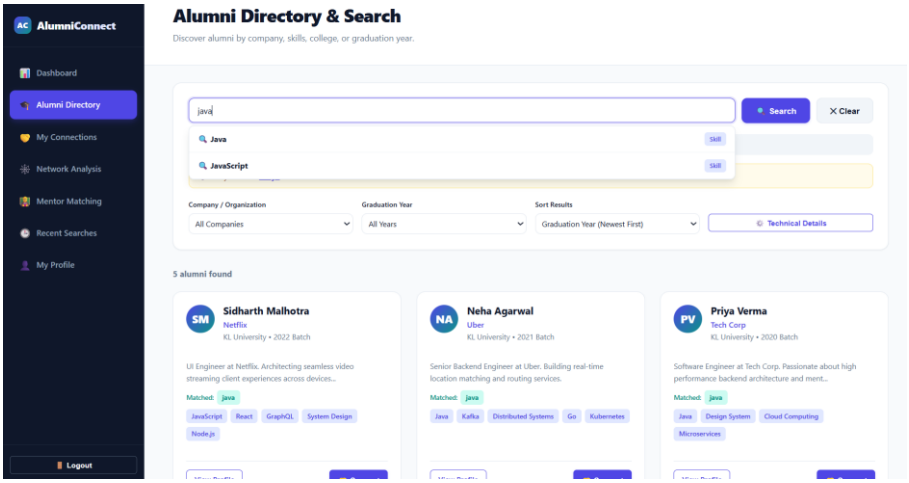


Figure A.5: Alumni directory search with autocomplete suggestions

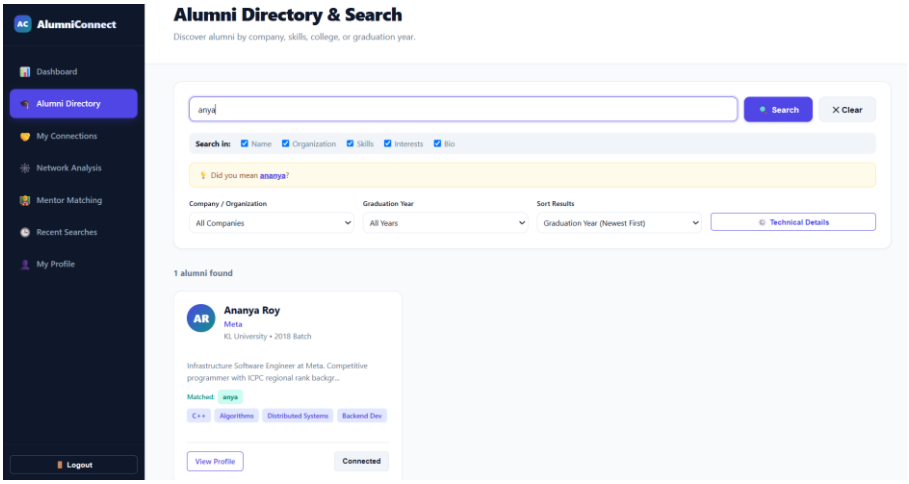


Figure A.6: Typo-correction suggestion in alumni search

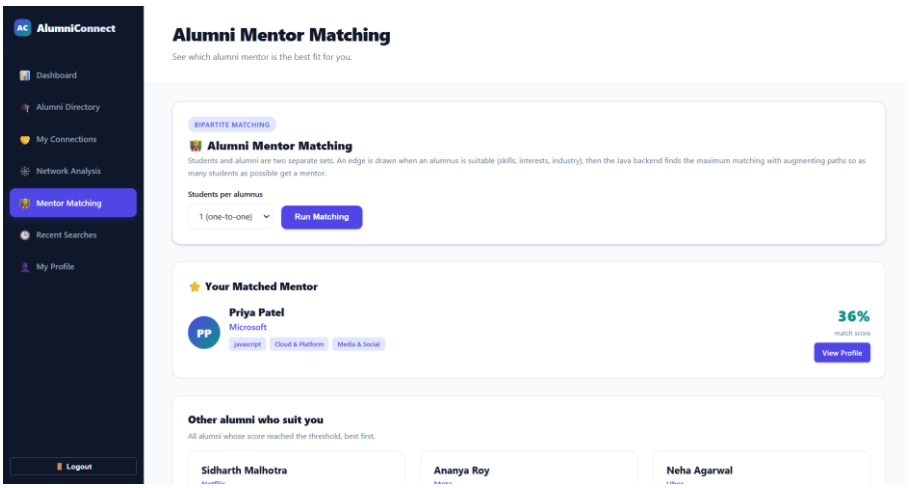


Figure A.7: Alumni mentor matching results